

Create DAX Calculations in Semantic Models

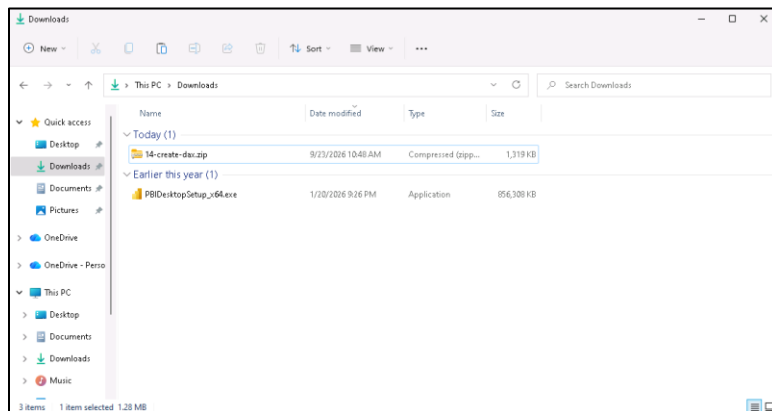
Certification Track: DP-600 [Fabric Analytics Engineer Associate]

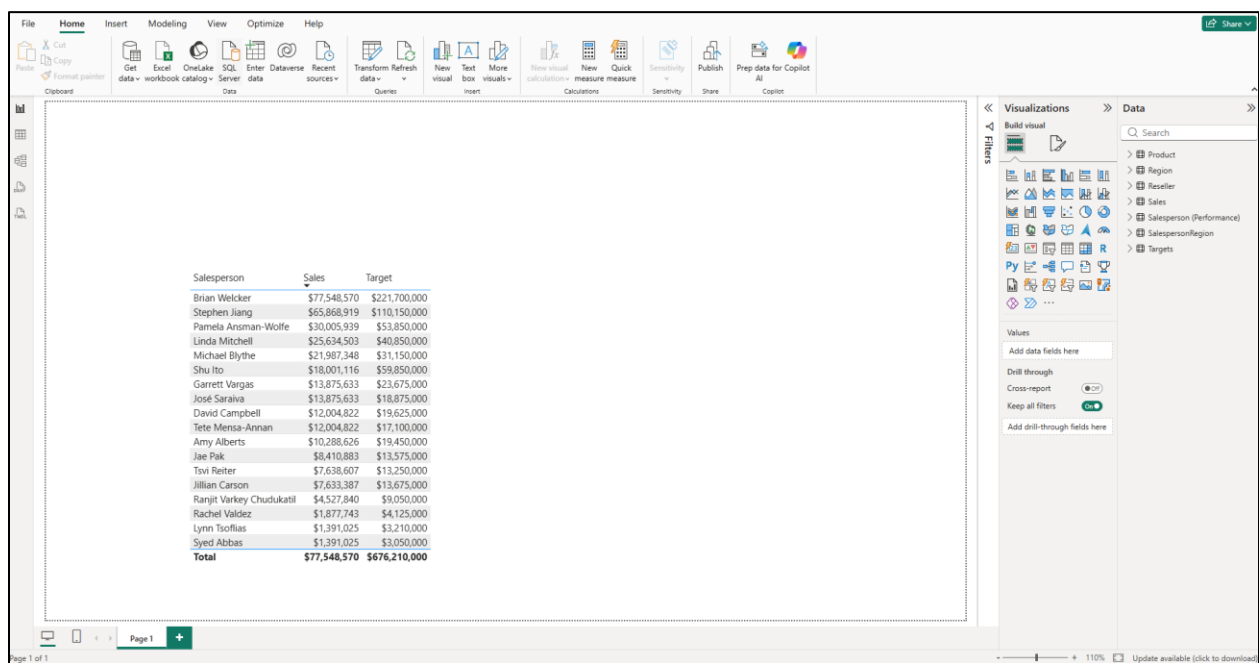
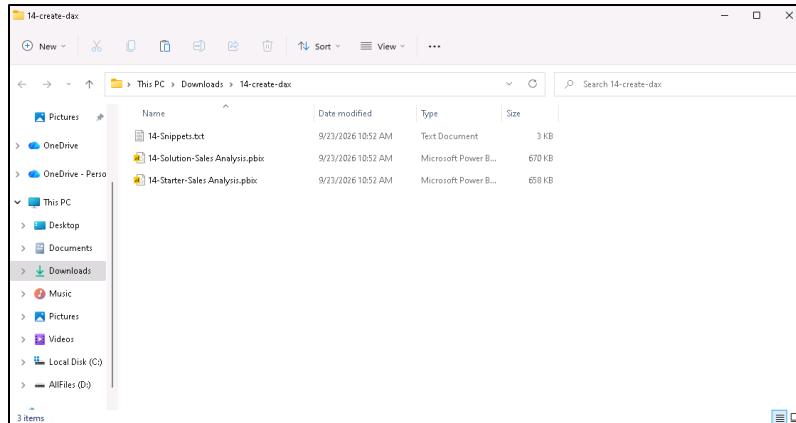
DAX originated from Excel, using familiar functions so Excel users can transition smoothly and quickly to writing DAX. Data Analysis Expressions (DAX) allows you to build powerful calculations in Power BI so you can analyze and visualize your data more effectively. DAX adds three types of calculations to a semantic model: Calculated Tables for creating structures like date tables, Calculated Columns for row-level logic, and Measures for aggregating and summarizing data across the model.

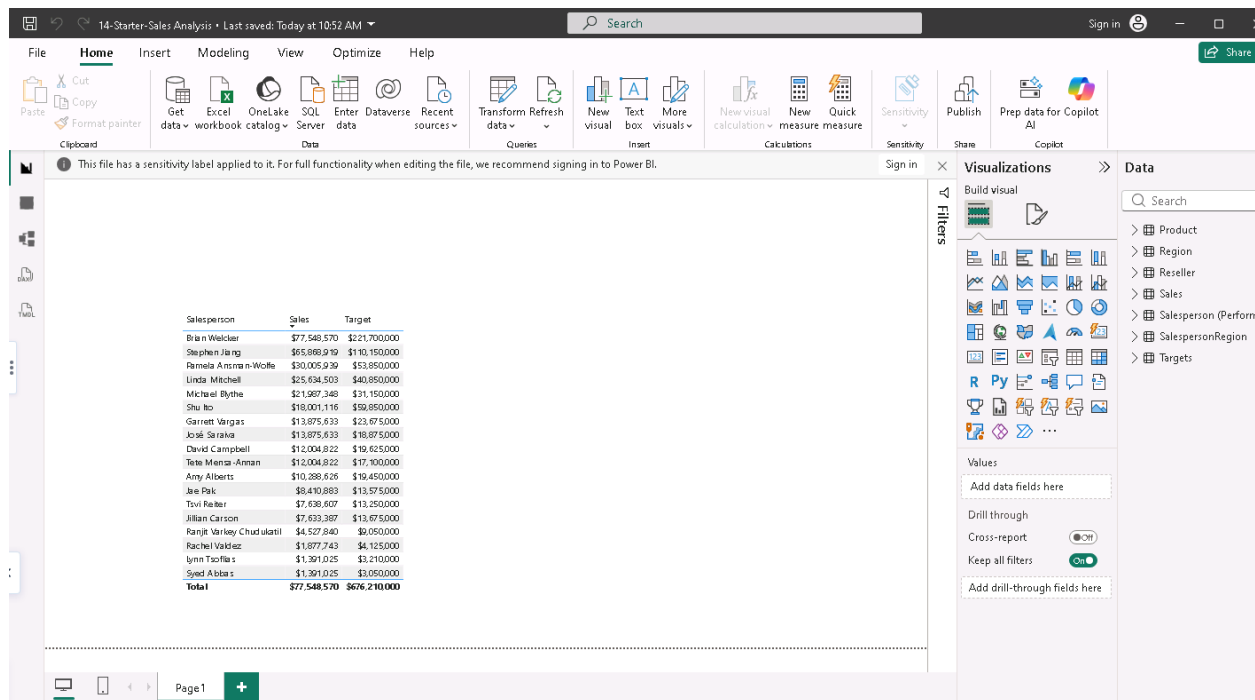
In this lab, you learn how to create calculated tables, create calculated columns, and create measures.

1. Get Started

- You need Power BI Desktop installed (November 2025 or newer).
- Open a web browser and go to URL:
<https://github.com/MicrosoftLearning/mslearn-fabric/raw/main/Allfiles/Labs/14/14-create-dax.zip>
- Download zip file.
- Extract the zip folder to the Downloads folder.
- Open the Power BI file named “14-Starter-Sales Analysis.pbix”.
- Dismiss any pop-ups by closing windows if appear, select Cancel, or select Apply Later





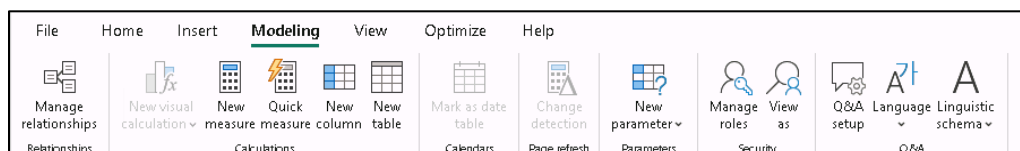


2. Create the Salesperson Calculated Table

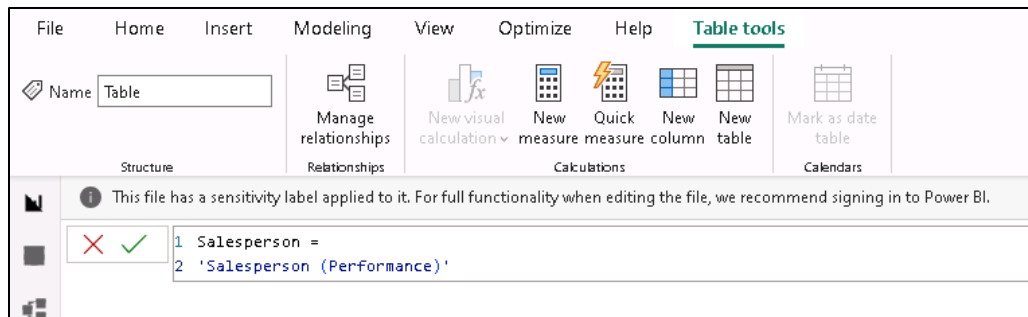
A calculated table is created by writing a DAX formula that returns a table and is stored in the model. This is useful for situations where you need a table that doesn't exist in your source data but is necessary for analysis or model design. You can use it to create a custom date table, combine or filter existing tables, and create reference or scenario-driven tables.

To create a calculated table, enter the table name, followed by the equals symbol (=), then a DAX formula that returns a table. The table name can't already exist in the data model.

- In Power BI Desktop, go Report view (bar graph icon on left column).
- On the Modeling ribbon, select New Table inside the Calculations group.



- In the formula bar type: Salesperson = 'Salesperson (Performance)' then press Enter or press on green checkmark to apply the calculation.
- This creates a copy of the Salesperson (Performance) table, but only the data is copied.

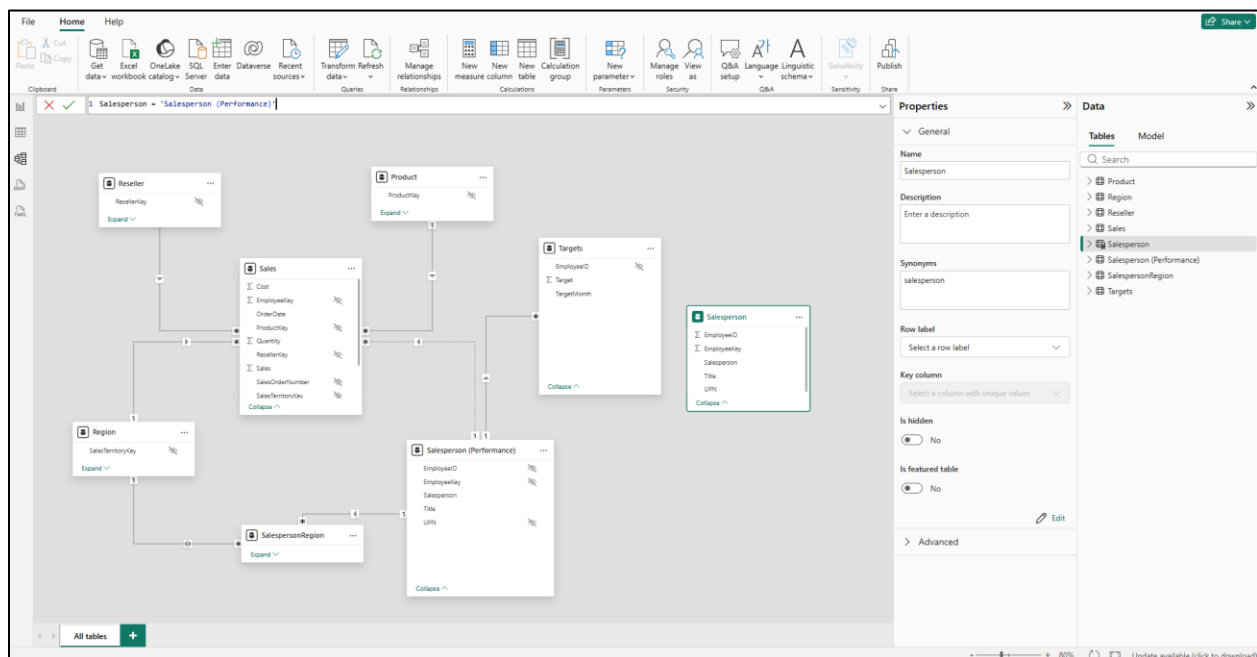


- In the Data pane to the far right, the table icon shows a calculator symbol indicating the Sales table is a calculated table.

Understanding Calculated Table:

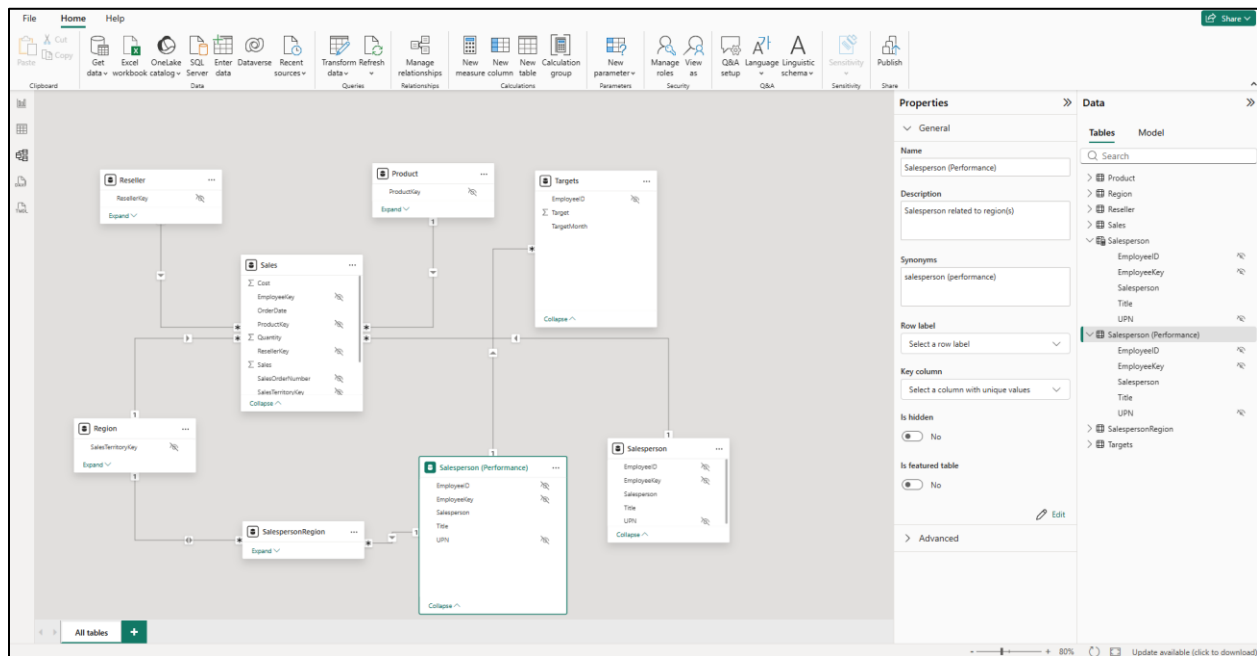
- Calculated tables store materialized data and increase model size.
- They refresh whenever dependent data changes.
- They cannot load external data, only transform data already in the model.

- Switch to model view (icon on left column) that shows the new Salesperson table.



- Drag Salesperson [EmployeeKey] column to the Sales [EmployeeKey] column to create a relationship. Make this relationship active by clicking Save.
- Remove the old inactive relationship: Right-click the inactive relationship between Salesperson(Performance) and Sales tables, Delete and click Yes.
- Hide unnecessary columns: in the Salesperson table, select them EmployeeID, EmployeeKey, UPI columns and set the Is Hidden to Yes.
- Add table descriptions:

- Select the Salesperson table, in Properties set Description to “Salesperson related to Sales”.
 - Select the Salesperson (Performance) table, in Properties set Description to “Salesperson related to region(s)”.
- Your model now supports two ways to analyze salespeople: Sales made by the salesperson (Salesperson table) and sales made in the salesperson’s assigned region (Salesperson (Performance) table).



3. Create the Date Table

A practical example of using a calculated table is creating a Date table for your model. The purpose of a Date table is to provide a complete and consistent calendar that DAX can use for filtering, grouping and time-intelligence calculations such as YTD. A Date table must not contain any blanks or missing dates, as this will cause calculations to fail or return incorrect or misleading results.

Follow the steps below to create a Date table using the CALENDARAUTO() function.

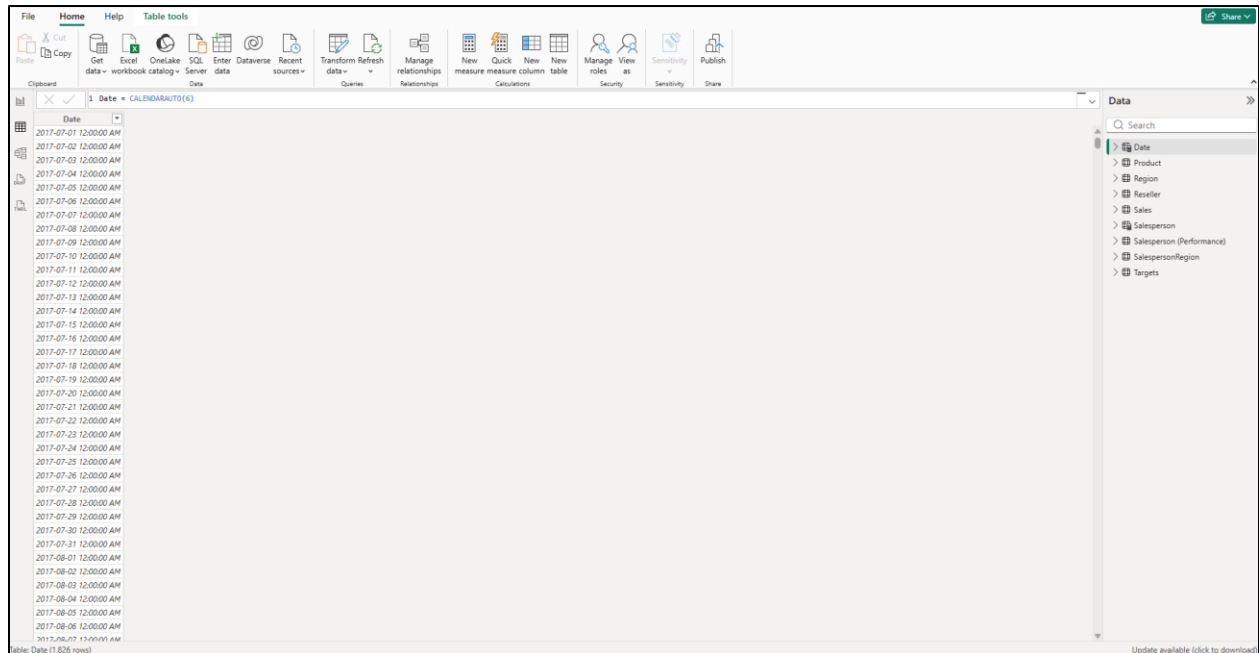
- Switch to Table view (grid icon on left column). On the Home tab select New Table inside the Calculations group.
- Enter the DAX formula in the bar: Date = CALENDARAUTO(6) and press Enter.

Understanding CALENDARAUTO() Function:

- Returns a single column table with date values.
- Scans your entire data model for any date columns and finds the earliest and latest dates in your data. This eliminates the need to manually define start and end dates.
- Builds a continuous list of dates covering full years.
- The number 6 tells Power BI that June is the last month of the year.

- If you don't specify a number, it defaults to 12 (December) as the last month of the year.

- Check the Date column: The Date values are formatted using US regional settings (mm/dd/yyyy).
- Confirm the number of rows: Look at the bottom left status bar that shows 1826 rows of data which is equivalent to five full years of dates.



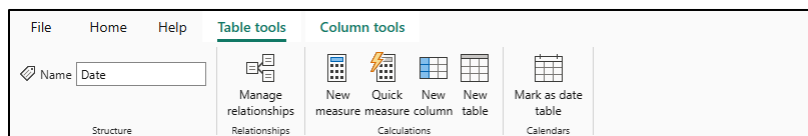
4. Create Calculated Columns

Calculated columns let you create fields that don't exist in your raw data such as concatenated text (FirstName + LastName). DAX formula is used to add a calculated column and returns a single value for each row, which is stored in your model.

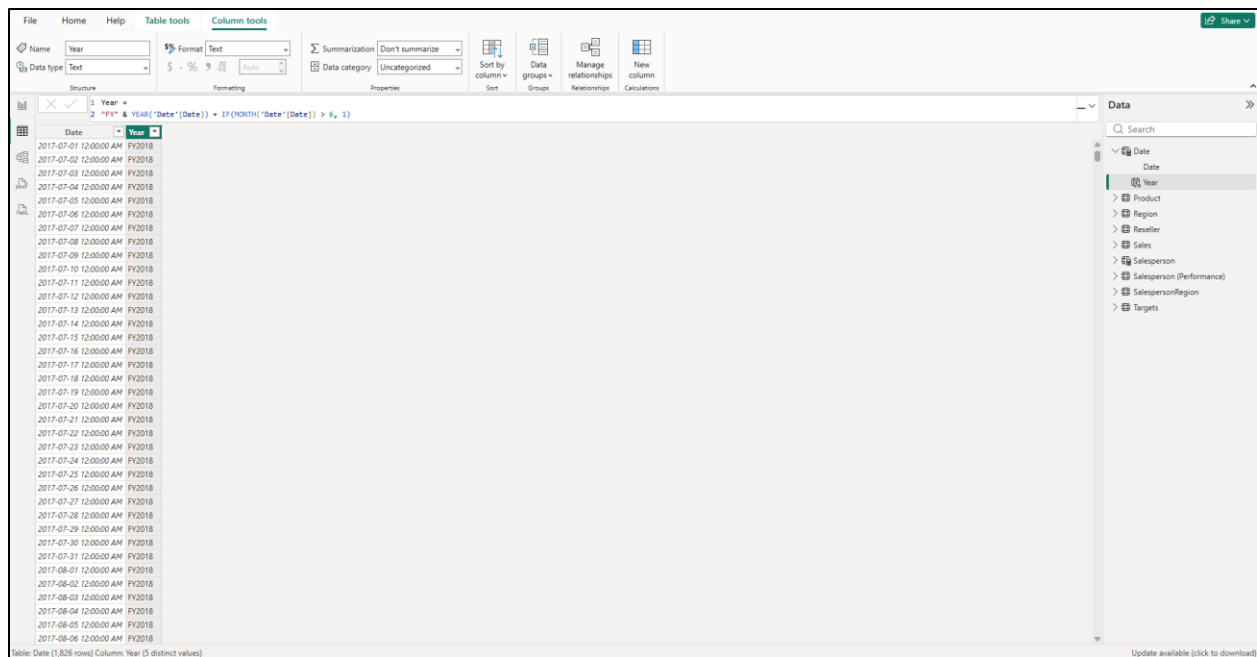
You will add more columns to demonstrate how filtering and grouping work across different time periods. Additionally, you'll create a calculated column to handle sorting.

Create new time-based columns

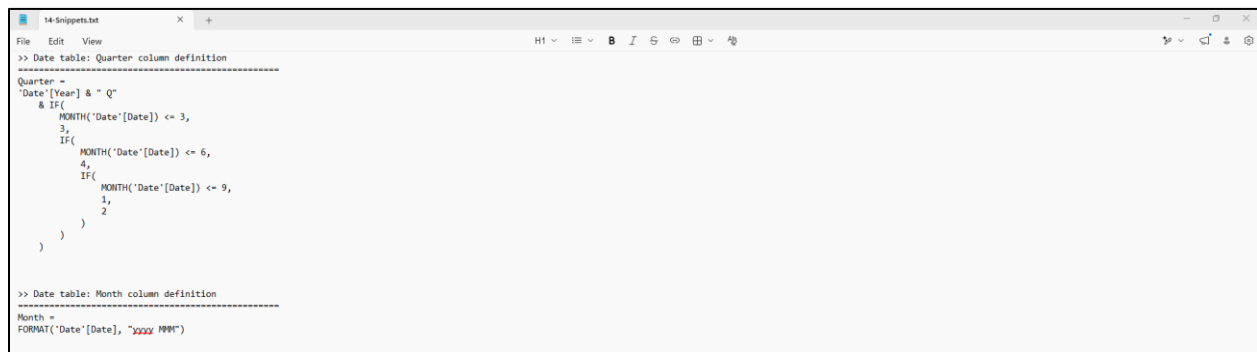
- Add a new calculated column: On the Table tools tab, select New Column inside the Calculations group.
- A calculated column is the column name, followed by the equals sign, followed by a DAX formula, and returns a single value. Column names must be unique.



- Create the Year column by entering the formula in the bar:
`Year = "FY" & YEAR('Date'[Date]) + IF(MONTH('Date'[Date]) > 6, 1)` and press Enter.
 - This creates fiscal years(FY).
 - Months after June belong to the next fiscal year.



- Create the Quarter and Months columns by using the definitions from Snippets.txt below.
- Copy the code block for the Quarter column and paste it into the formula bar. Repeat this for the Month column.
- Confirm the new columns appear in the table.



Quarter column created:

The screenshot shows the Power BI Desktop interface. The 'Table tools' ribbon is active, and the 'Column tools' group is visible. The 'Structure' pane on the left shows the 'Date' table. The 'Data' pane on the right shows the 'Date' table with columns: Date, Year, Quarter, and Targets. The 'Quarter' column is highlighted.

DAX Formula:

```
1 Quarter =  
2 IF(  
3     & "Q"  
4     MONTH('Date'[Date]) <= 3,  
5     1,  
6     IF(  
7         MONTH('Date'[Date]) <= 6,  
8         2,  
9         IF(  
10            MONTH('Date'[Date]) <= 9,  
11            3,  
12            4  
13        )  
14    )  
15 )  
16
```

Data Table:

Date	Year	Quarter
2017-07-01 12:00:00 AM	FY2018	FY2018 Q1
2017-07-02 12:00:00 AM	FY2018	FY2018 Q1
2017-07-03 12:00:00 AM	FY2018	FY2018 Q1
2017-07-04 12:00:00 AM	FY2018	FY2018 Q1
2017-07-05 12:00:00 AM	FY2018	FY2018 Q1
2017-07-06 12:00:00 AM	FY2018	FY2018 Q1
2017-07-07 12:00:00 AM	FY2018	FY2018 Q1
2017-07-08 12:00:00 AM	FY2018	FY2018 Q1
2017-07-09 12:00:00 AM	FY2018	FY2018 Q1
2017-07-10 12:00:00 AM	FY2018	FY2018 Q1
2017-07-11 12:00:00 AM	FY2018	FY2018 Q1
2017-07-12 12:00:00 AM	FY2018	FY2018 Q1
2017-07-13 12:00:00 AM	FY2018	FY2018 Q1
2017-07-14 12:00:00 AM	FY2018	FY2018 Q1
2017-07-15 12:00:00 AM	FY2018	FY2018 Q1
2017-07-16 12:00:00 AM	FY2018	FY2018 Q1
2017-07-17 12:00:00 AM	FY2018	FY2018 Q1
2017-07-18 12:00:00 AM	FY2018	FY2018 Q1
2017-07-19 12:00:00 AM	FY2018	FY2018 Q1
2017-07-20 12:00:00 AM	FY2018	FY2018 Q1
2017-07-21 12:00:00 AM	FY2018	FY2018 Q1
2017-07-22 12:00:00 AM	FY2018	FY2018 Q1
2017-07-23 12:00:00 AM	FY2018	FY2018 Q1
2017-07-24 12:00:00 AM	FY2018	FY2018 Q1
2017-07-25 12:00:00 AM	FY2018	FY2018 Q1

Table: Date (1,828 rows) Columns: Quarter (25 distinct values)

Month column created:

The screenshot shows the Power BI Desktop interface. The 'Table tools' ribbon is active, and the 'Column tools' group is visible. The 'Structure' pane on the left shows the 'Date' table. The 'Data' pane on the right shows the 'Date' table with columns: Date, Year, Quarter, Month, and Targets. The 'Month' column is highlighted.

DAX Formula:

```
1 Month =  
2 FORMAT('Date'[Date], "yyyy MM")  
3
```

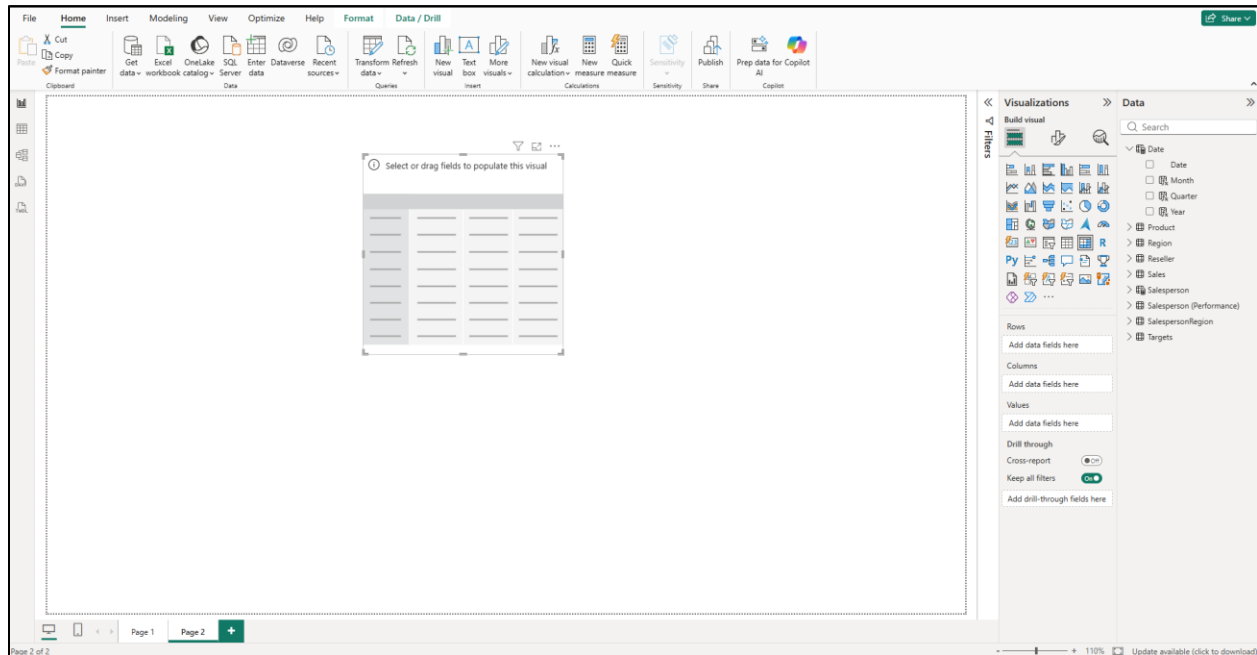
Data Table:

Date	Year	Quarter	Month
2017-07-01 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-02 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-03 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-04 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-05 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-06 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-07 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-08 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-09 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-10 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-11 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-12 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-13 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-14 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-15 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-16 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-17 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-18 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-19 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-20 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-21 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-22 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-23 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-24 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-25 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-26 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-27 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-28 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-29 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-30 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-07-31 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
2017-08-01 12:00:00 AM	FY2018	FY2018 Q1	2017 Aug
2017-08-02 12:00:00 AM	FY2018	FY2018 Q1	2017 Aug
2017-08-03 12:00:00 AM	FY2018	FY2018 Q1	2017 Aug
2017-08-04 12:00:00 AM	FY2018	FY2018 Q1	2017 Aug
2017-08-05 12:00:00 AM	FY2018	FY2018 Q1	2017 Aug

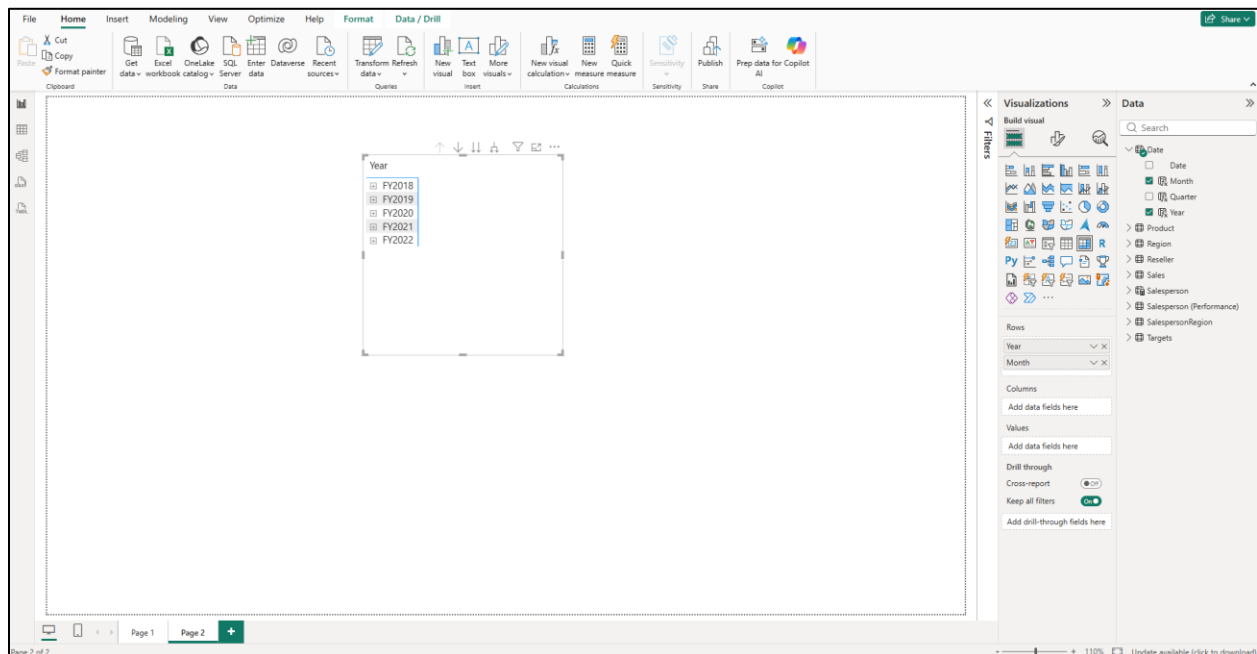
Table: Date (1,828 rows) Columns: Month (80 distinct values)

Validate in Report View

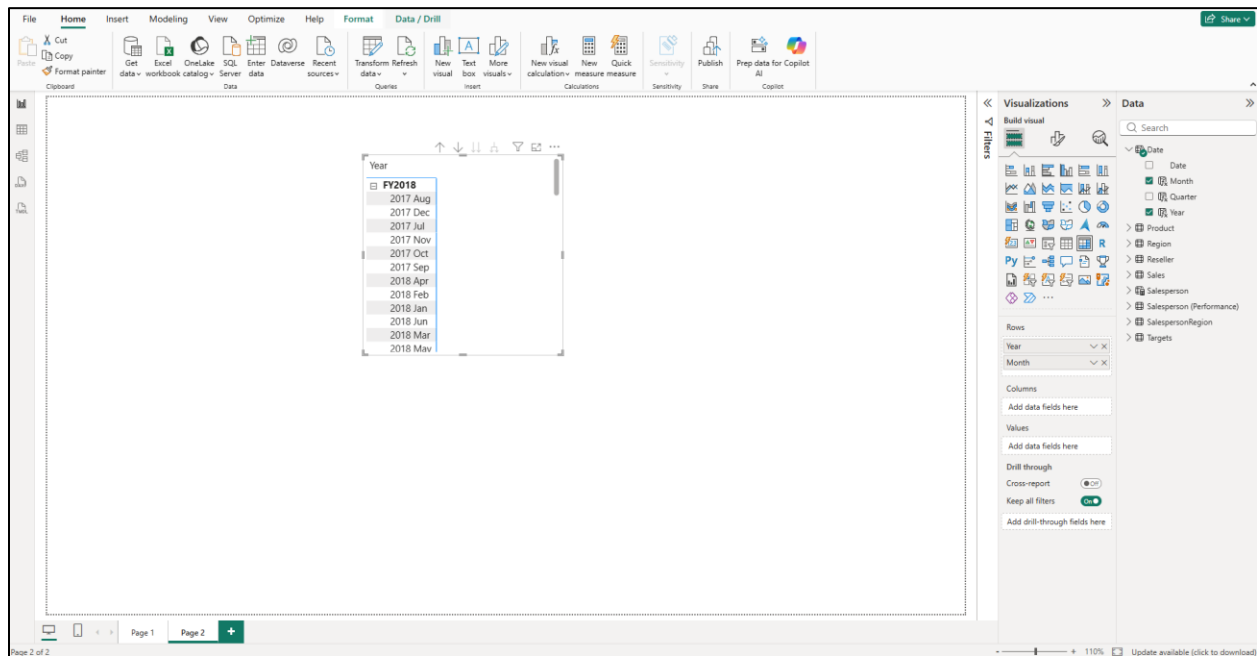
- Switch to Report view (bar graph icon on left column)
- Select the + icon next to Page 1 at the bottom left of the screen.
- Add a matrix visual: In the Visualizations pane, choose the Matrix icon.



- Add fields to the matrix: From the Date table, drag Year field into Rows text box. Drag Month field into Rows (under Year).



- Expand the hierarchy: On the matrix visual, select the forked-double arrow icon.
- This expands all the years to show the months. Months are sorted in alphabetical order instead of calendar order.



Fix month sorting

- Switch to Table view.
- Add a new column called MonthKey to the Date table by entering in the formula bar:

$$\text{MonthKey} = (\text{YEAR}('Date'[Date]) * 100) + \text{MONTH}('Date'[Date])$$
- The new column creates numeric values like 201707 for July 2017.

Date	Year	Quarter	Month	MonthKey
2017-07-01 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-02 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-03 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-04 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-05 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-06 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-07 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-08 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-09 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-10 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-11 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-12 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-13 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-14 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-15 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-16 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-17 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-18 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-19 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-20 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-21 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-22 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-23 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-24 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-25 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-26 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-27 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-28 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-29 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-30 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-07-31 12:00:00 AM	FY2018	Q1	2017 Jul	201707
2017-08-01 12:00:00 AM	FY2018	Q1	2017 Aug	201708
2017-08-02 12:00:00 AM	FY2018	Q1	2017 Aug	201708
2017-08-03 12:00:00 AM	FY2018	Q1	2017 Aug	201708
2017-08-04 12:00:00 AM	FY2018	Q1	2017 Aug	201708
2017-08-05 12:00:00 AM	FY2018	Q1	2017 Aug	201708
2017-08-06 12:00:00 AM	FY2018	Q1	2017 Aug	201708

- Switch back to Report view.
- Select the Month field in the Data pane.
- On the Column tools tab, select Sort by Column, then select MonthKey.

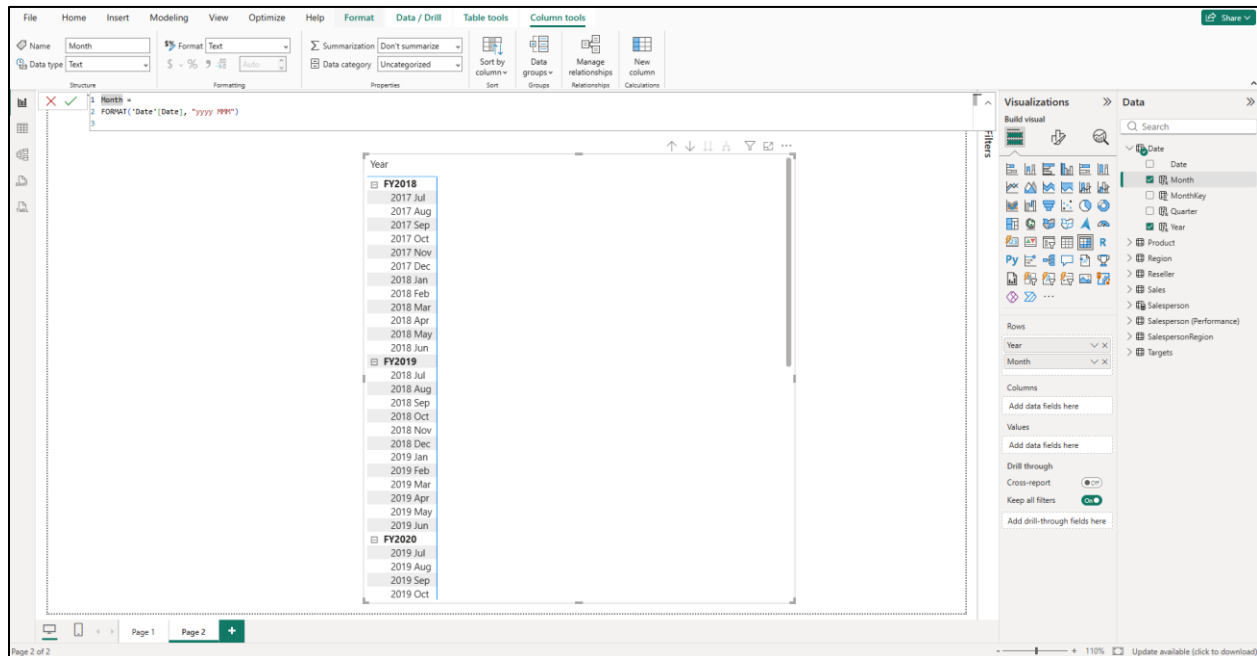
Month =

FORMAT('Date'[Date], "yyyy MMM")

Sort by column

- Month
- Date
- MonthKey
- Quarter
- Year

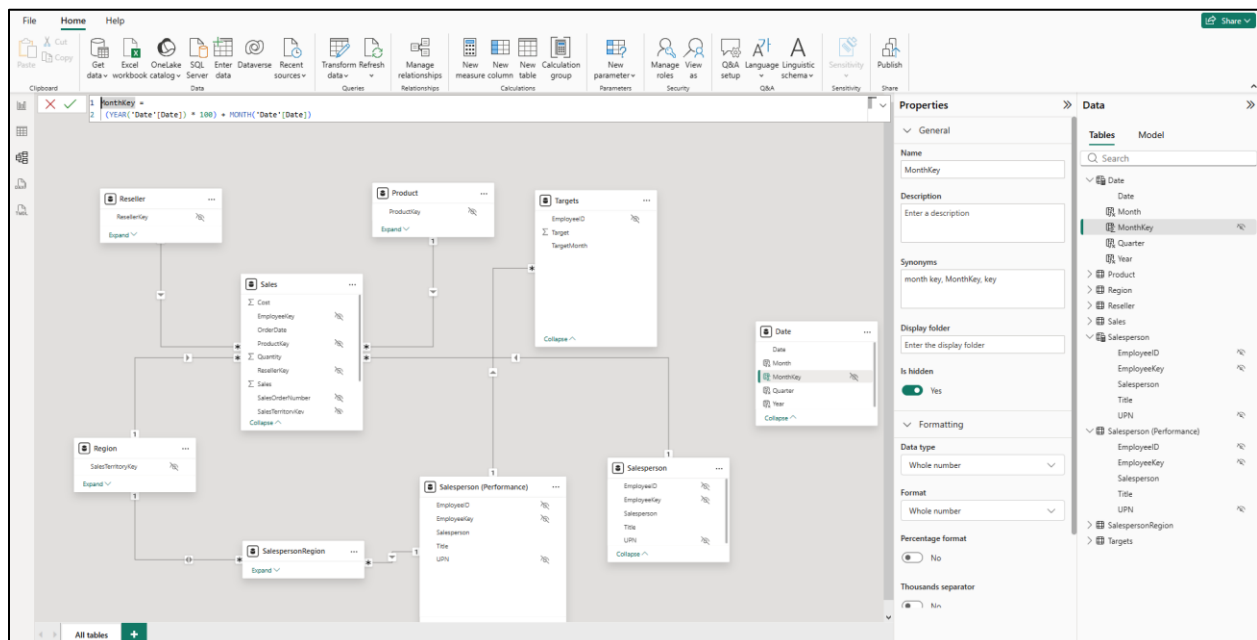
- Check the matrix visual: The Months now appear in chronological order.



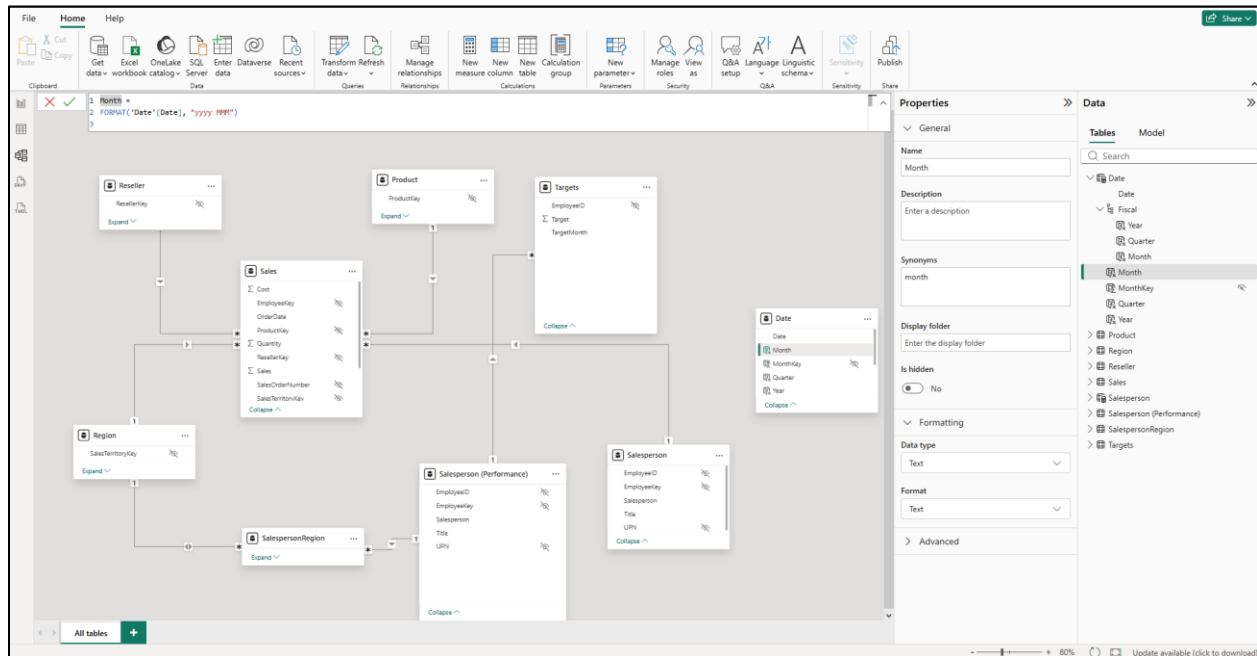
5. Complete the Date table

To finish the design of the Date table, you will hide a column, create a hierarchy, and create relationships to the Sales and Targets tables.

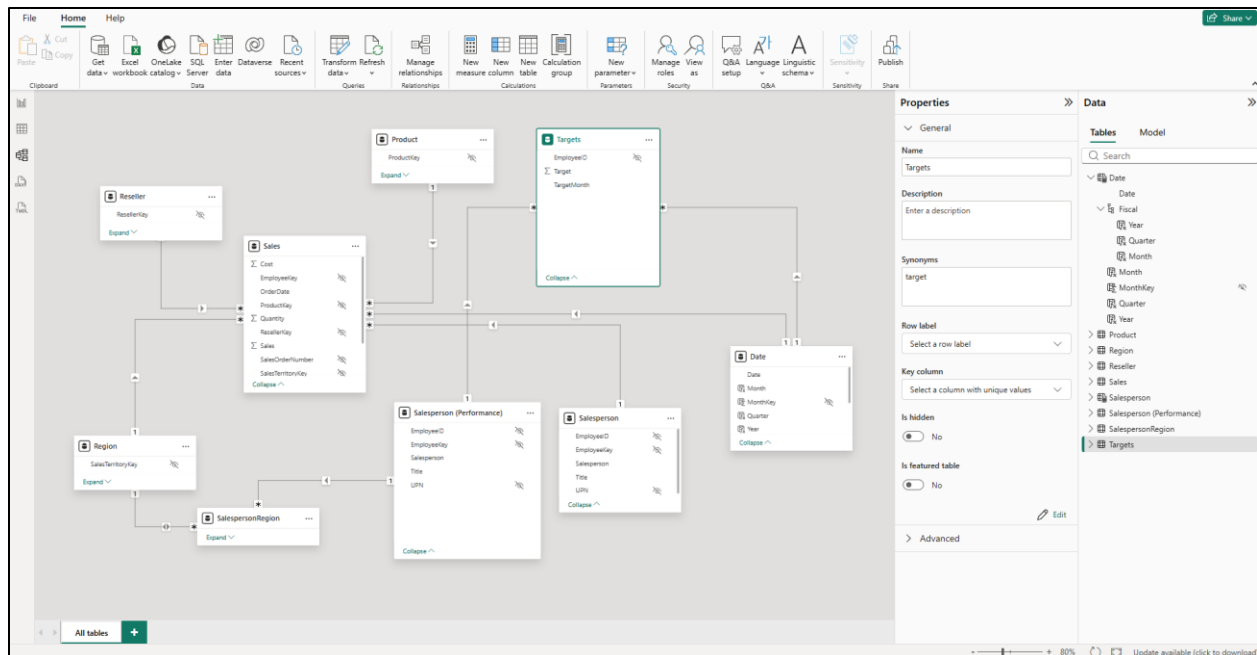
- Hide a column: Switch to Model view. In the Date table select MonthKey. Set Is Hidden to Yes.



- Create the Fiscal hierarchy: In the Data pane, select the Date table.
 - Right click Year column and select Create hierarchy.
 - Right click the new hierarchy and rename “Fiscal”.
 - Right click Quarter column, Add to hierarchy Fiscal.
 - Right click Month column, Add to hierarchy Fiscal.



- Create two model relationships:
 - Drag the Date column (Date table) to the OrderDate column (Sales table).
 - Drag the Date column (Date table) to the TargetMonth column (Targets table).



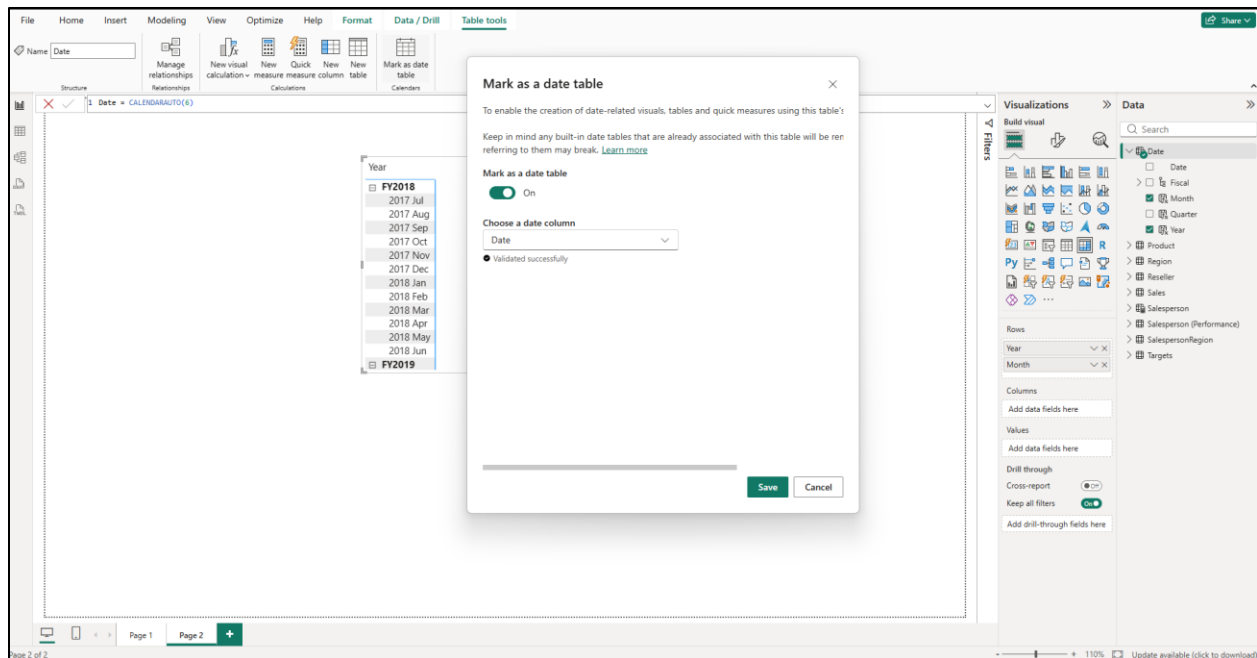
- Hide two columns in related tables:
 - In the Sales table, select the eye icon to the right of the OrderDate field. Notice there is a line crossed through the eye and Is Hidden is set to Yes. Either way will hide the column.
 - Repeat for the Targets table (TargetMonth column).

6. Mark the Date table

Mark as Date Table tells Power BI which table is your main date table- the official calendar for all date calculations. It enables proper time intelligence functions (YTD, MTD, YOY) and ensures date filters work correctly.

Once you have created a date table, you need to Mark it as a date table.

- Switch to Report view.
- In the Data pane, select the Date table (NOT the Date column).
- On the Table tools tab, select Mark as Date Table.
- In the Mark as a Date Table window:
 - Turn Mark as a Date Table to On.
 - In Choose a date column, select Date.
 - Select Save.
- Save your Power BI Desktop file.

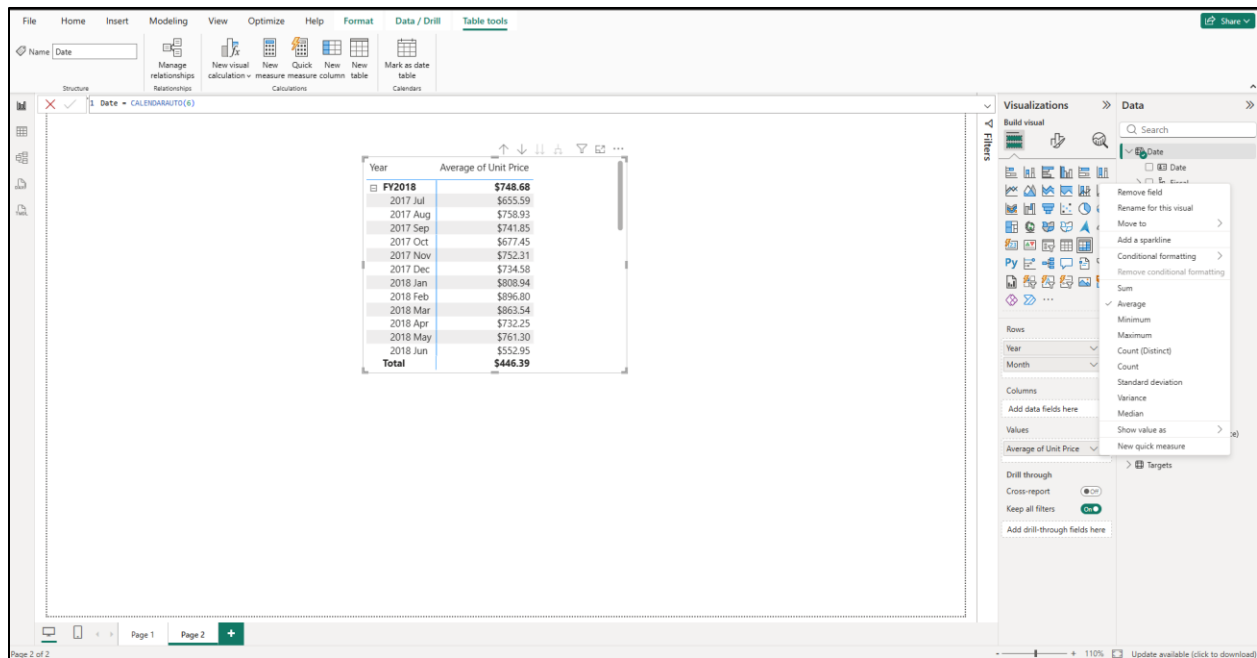


7. Create Simple Measures

A measure always returns one value and never stores data in the model. Measures are calculated each time the report runs a query. Simple measures aggregate values in a single column or count rows in a table without writing any DAX. Simple measures are quick, automatic calculations Power BI creates that work well for fast visuals and smaller datasets. Power BI lets report users change how a field is summarized, giving them the flexibility to choose the aggregation that best fits their needs.

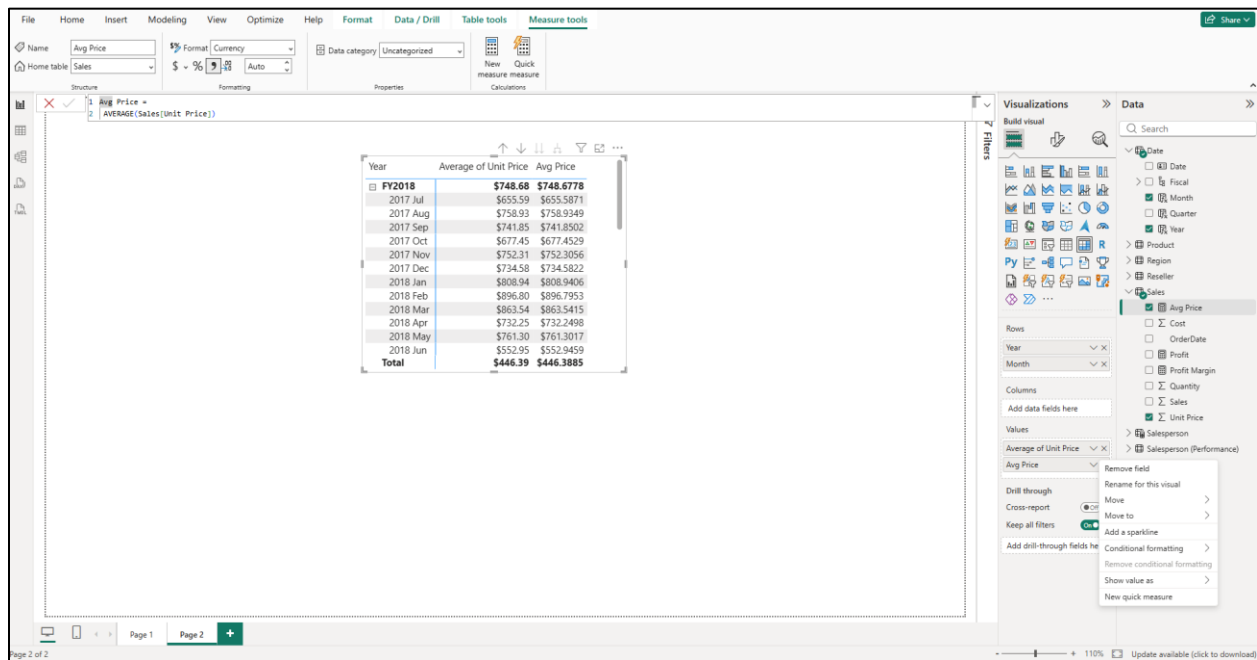
Add Unit Price to visual:

- Go to Report view and make sure you are on Page 2.
- In the Data pane, drag the Unit Price field (Sales table) into the matrix visual.
- Check the default aggregation: In the Values area, you will see Average of Unit Price. Average is the default aggregation, but you can click on the dropdown to view other aggregation options like Sum, Minimum, Maximum, Count.



Create the Avg Price Measure:

- In the Data pane, right click the Sales table and select New Measure.
- Enter in the formula bar: Avg Price = AVERAGE(Sales[Unit Price])
- Drag Avg Price measure into the matrix visual. It shows the same result as the Unit Price column but with different formatting.
- Notice you can't change its aggregation when you click on the drop down beside Avg Price. This is because measure have fixed behaviour and cannot be modified.



Create the Remaining Measures:

- Use your 14-Snippet.txt file to create the five measures for the Sales table:
 - Median Price
 - Min Price
 - Max Price
 - Orders (counts each order once)
 - Order Lines (counts every row)
- Remember: Right-click the Sales table and click New Measure. Then enter the code in the formula bar.

```

14-Snippets.txt
File Edit View
H1
-----
MonthKey =
(YEAR("Date"[Date]) * 100) + MONTH("Date"[Date])

>> Sales table: Avg Price measure definition
-----
Avg Price =
AVERAGE(Sales[Unit Price])

>> Sales table: Additional measure definitions (use one definition at a time)
-----
Median Price =
MEDIAN(Sales[Unit Price])

Min Price =
MIN(Sales[Unit Price])

Max Price =
MAX(Sales[Unit Price])

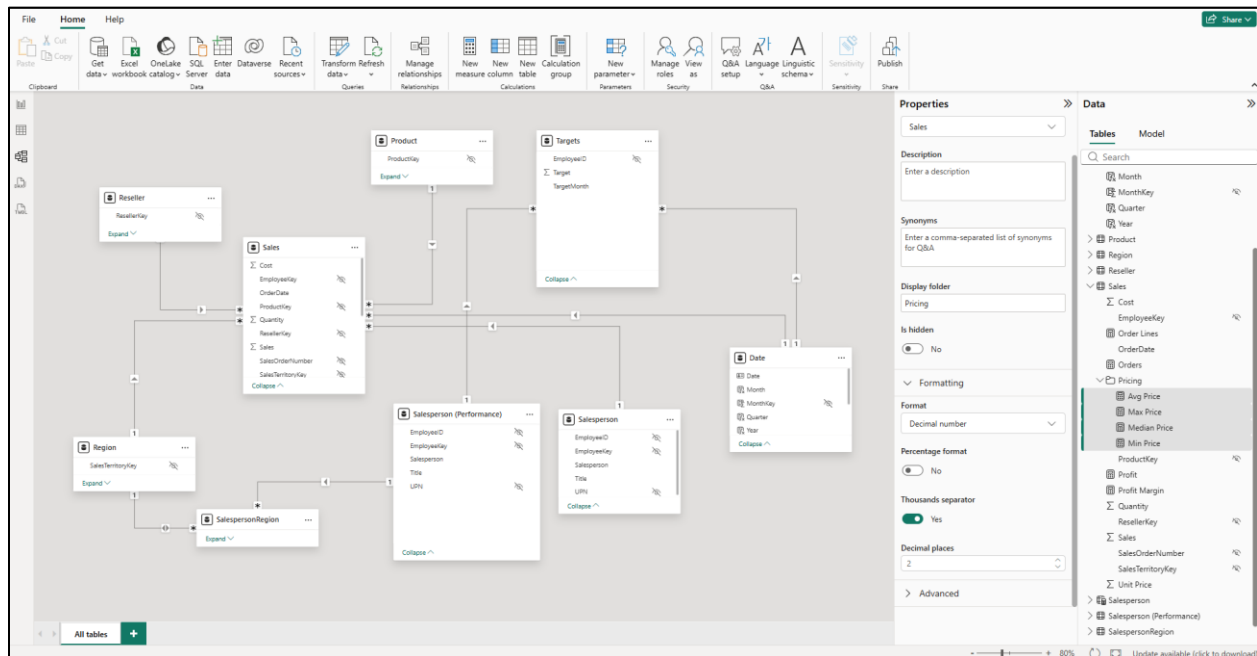
Orders =
DISTINCTCOUNT(Sales[SalesOrderNumber])

Order Lines =
COUNTROWS(Sales)

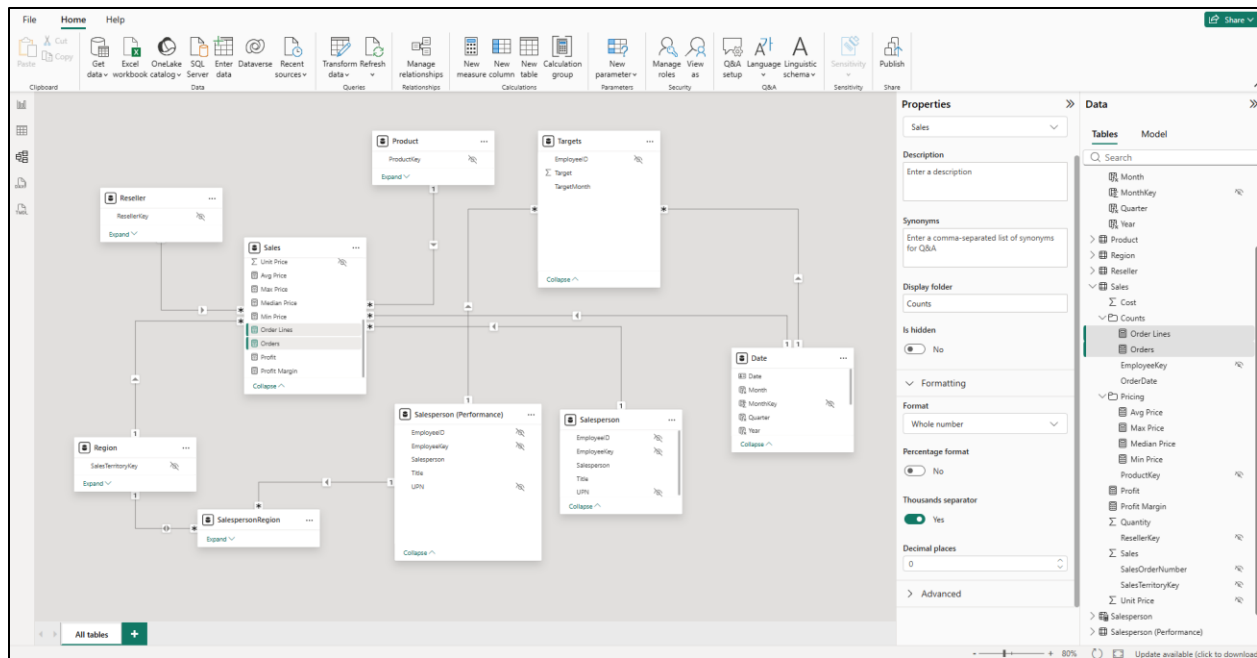
```

Format and *Organize the Measures:

- Switch to Model view.
- Multi-select the four price measures: Avg Price, Max Price, Median Price, Min Price.
- Set Format to 2 decimal places.
- Set Display Folder to “Pricing”.



- Hide the Unit Price Column: Select the Unit Price column in the Sales table and set Is Hidden to Yes.
- Format the count measure: Multi-select Orders and Order Lines measures.
- Set Format to Use thousands separator.
- Set Display Folder to “Counts”.



Update the Matrix Visual:

- Remove the raw column: First, switch to Report view.
- In the matrix Values area, remove Unit Price by selecting X.
- Resize the visual: Increase the size of the visual by dragging the corner to fill the page.
- Add all five measures to the matrix visual: Median Price, Min Price, Max Price, Orders, Order Lines.
- Verify results: Check that values and formatting look correct.

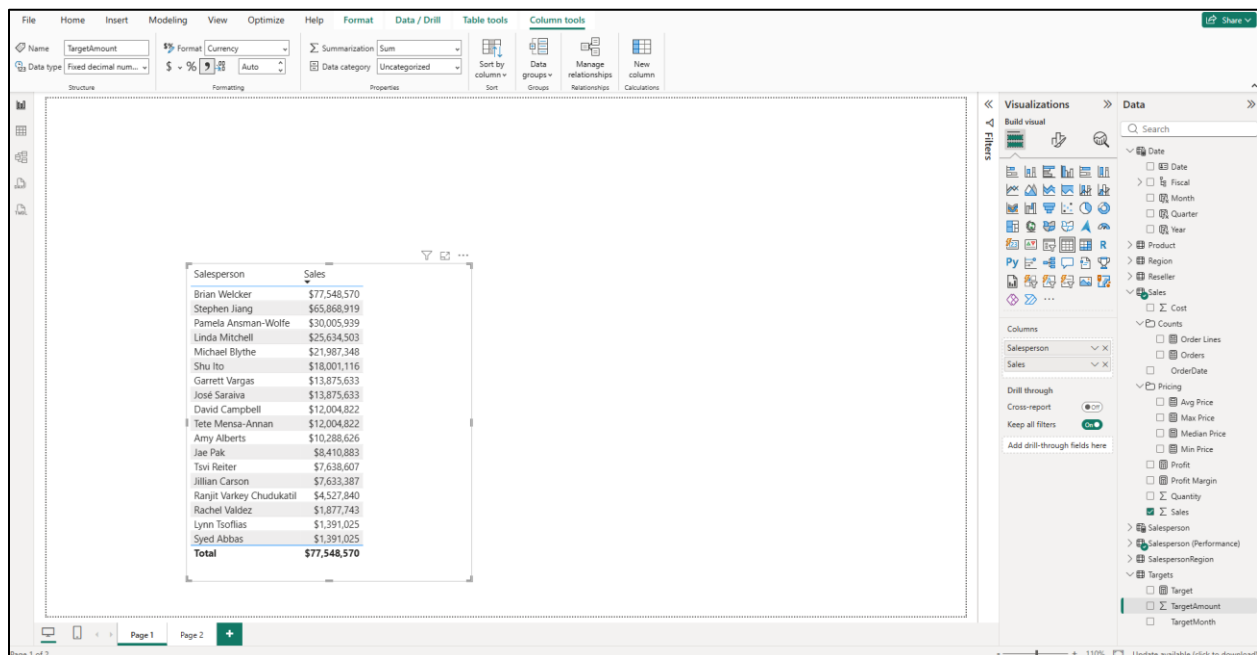
Year	Avg Price	Max Price	Min Price	Median Price	Orders	Order Lines
FY2018	748.68	2,146.96	4.75	419.46	739	8,459
2017 Jul	655.59	2,146.96	5.19	419.46	38	352
2017 Aug	758.93	2,146.96	4.75	419.46	75	785
2017 Sep	741.85	2,146.96	5.19	419.46	60	593
2017 Oct	677.45	2,146.96	5.19	419.46	40	499
2017 Nov	752.31	2,146.96	5.01	419.46	90	1,106
2017 Dec	734.58	2,146.96	5.01	419.46	63	803
2018 Jan	808.94	2,146.96	5.19	419.46	40	377
2018 Feb	896.80	2,146.96	5.01	419.46	79	866
2018 Mar	863.54	2,146.96	5.19	419.46	64	653
2018 Apr	732.25	2,146.96	5.19	419.46	37	494
2018 May	761.30	2,146.96	4.75	419.46	85	1,112
2018 Jun	552.95	2,146.96	4.75	419.46	68	819
FY2019	397.81	1,466.01	4.32	202.33	1,255	21,670
2019 Feb	488.75	1,466.01	5.19	469.79	132	1,664
2019 Jan	428.87	1,466.01	5.19	324.45	65	983
2019 Mar	450.03	1,466.01	5.01	324.45	106	1,215
2018 Nov	412.88	1,466.01	5.01	209.26	133	2,345
2018 Aug	372.75	1,466.01	4.75	202.33	139	2,964
2018 Oct	382.00	1,466.01	5.19	202.33	73	1,406
2018 Dec	407.08	1,466.01	4.75	202.33	114	1,732
2019 May	400.51	1,466.01	4.75	202.33	134	2,306
2019 Jun	390.32	1,466.01	4.75	202.33	102	1,698
2019 Apr	369.78	1,466.01	5.01	198.04	74	1,449
2018 Jul	350.74	1,466.01	4.75	196.33	72	1,723
2018 Sep	365.30	1,466.01	4.32	196.33	111	2,185
FY2020	392.12	1,466.01	1.33	200.05	1,622	27,722
2020 Feb	477.84	1,466.01	1.37	323.99	174	2,076
2020 Mar	485.27	1,466.01	1.37	323.99	174	2,177
2020 Jan	433.50	1,466.01	1.37	242.99	94	1,382
2019 Dec	412.52	1,466.01	1.37	202.33	176	3,029
2019 Nov	395.80	1,466.01	1.37	200.05	178	2,904
2019 Jul	350.55	1,466.01	1.37	158.43	94	2,186
2019 Sep	343.40	1,466.01	1.37	158.43	176	3,760
Total	446.39	2,146.96	1.33	214.24	3,616	57,851

8. Create Additional Measures

Simple measures only work for simple cases and can't handle complex calculations. Complex DAX measures go beyond basic aggregation to give you more control and produce dynamic metrics that respond to slicers, filters, and business context.

You will create measures that use more complex formulas.

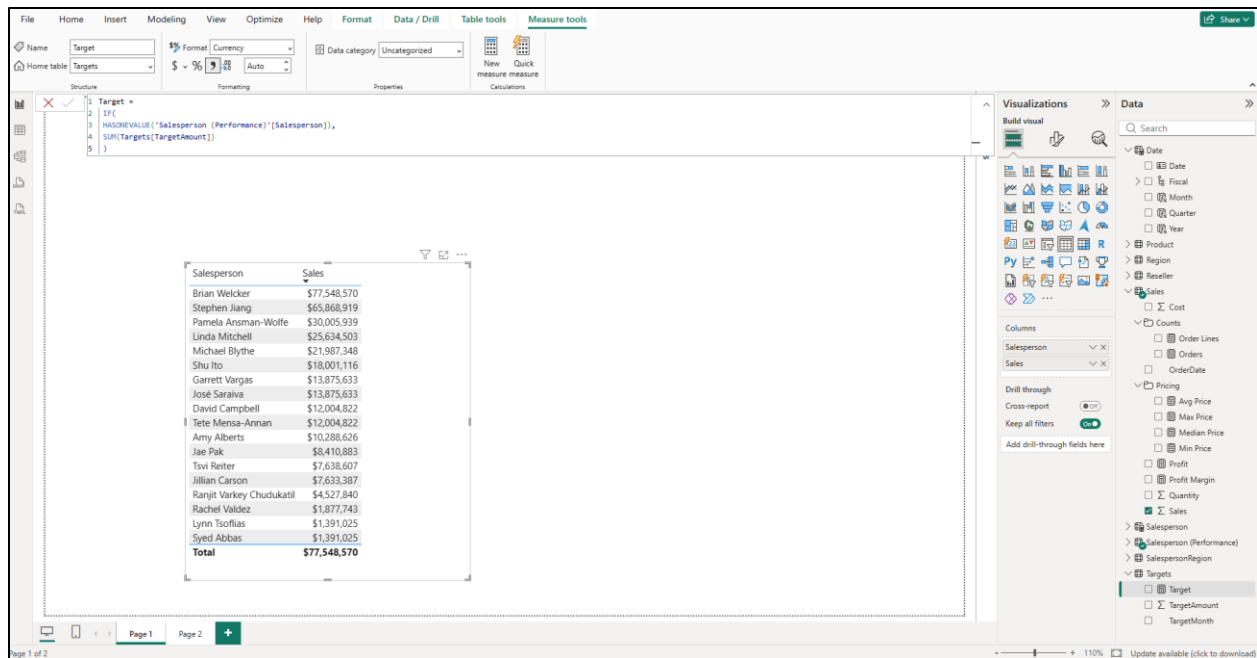
- Review the existing table: Go to Report and select Page 1. Look at the table visual and notice the Target column total.
- Remove the old Target column: Select the table visual. In the Visualizations pane, remove the Target column by clicking on the X beside it.
- Rename the Target column to TargetAmount: In the Data pane under the Targets table, right-click the Target column, select Rename, and type "TargetAmount".



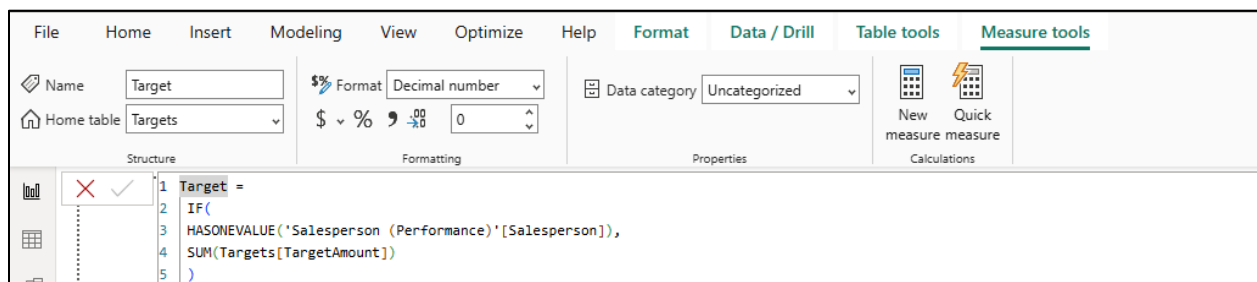
- Create the Target measure: Right-click on the Targets table, select New Measure and enter the code in the formula bar.

```
Target =  
IF(  
  HASONEVALUE('Salesperson (Performance)'[Salesperson]),  
  SUM(Targets[TargetAmount])  
)
```

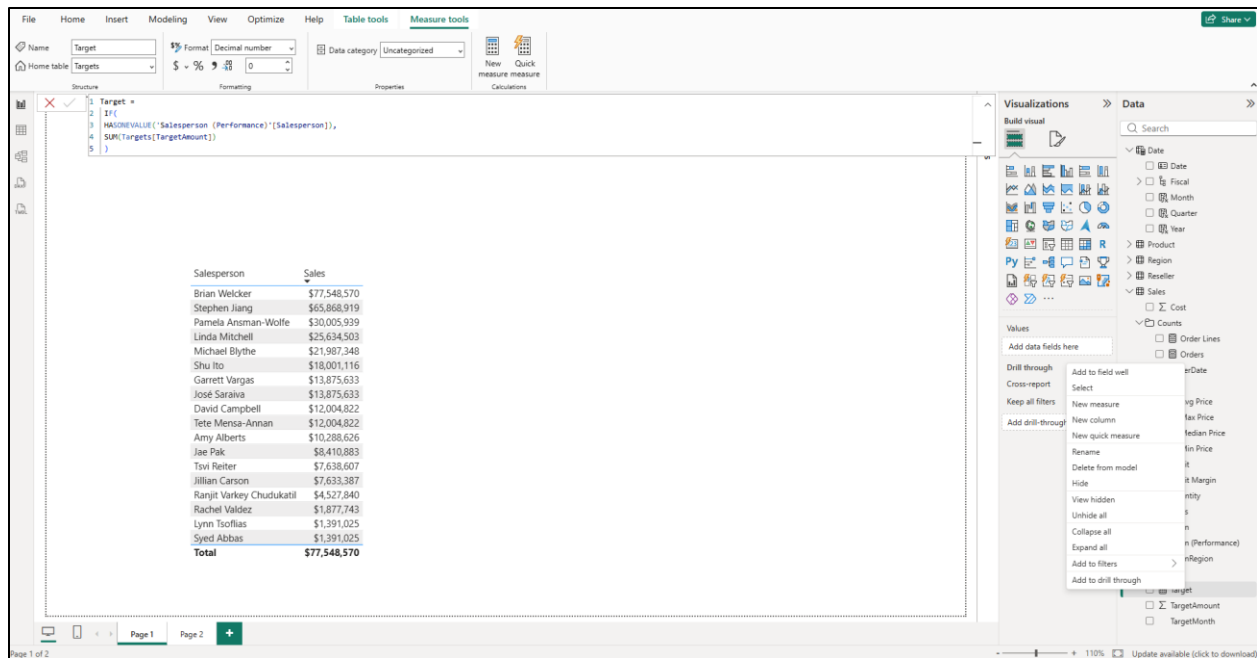
- The HASONEVALUE() function returns the salesperson's target when one salesperson is selected. When no single salesperson is selected, it returns BLANK.



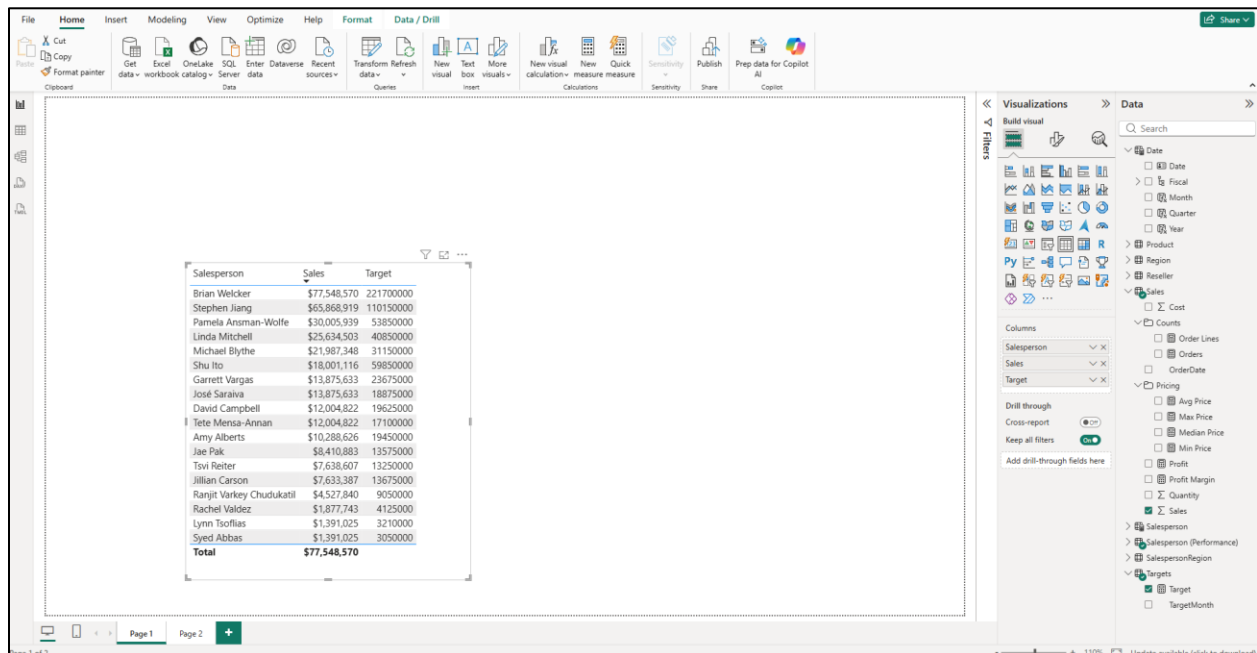
- Format the Target measure: Select the Measure Tools tab. From the Format drop down menu, select Decimal number. Change the Decimal place value to 0.



- Hide the TargetAmount column: In the Data pane, right-click on TargetAmount and select Hide.



- Add the Target measure to the visual: From the Data pane under the Targets table, drag the Target measure to the visual.
- Notice the Total for Target now shows BLANK.



- Create Variance and Variance Margin measures: Use your 14-Snippet.txt file to create two measures for the Targets table. Right-click on the Targets table, select New Measure and add the code in the formular bar.

```

14-Snippets.txt
File Edit View
-----
MonthKey =
(YEAR('Date'[Date]) * 100) + MONTH('Date'[Date])

>> Sales table: Avg Price measure definition
-----
Avg Price =
AVERAGE(Sales[Unit Price])

>> Sales table: Additional measure definitions (use one definition at a time)
-----
Median Price =
MEDIAN(Sales[Unit Price])

Min Price =
MIN(Sales[Unit Price])

Max Price =
MAX(Sales[Unit Price])

Orders =
DISTINCTCOUNT(Sales[SalesOrderNumber])

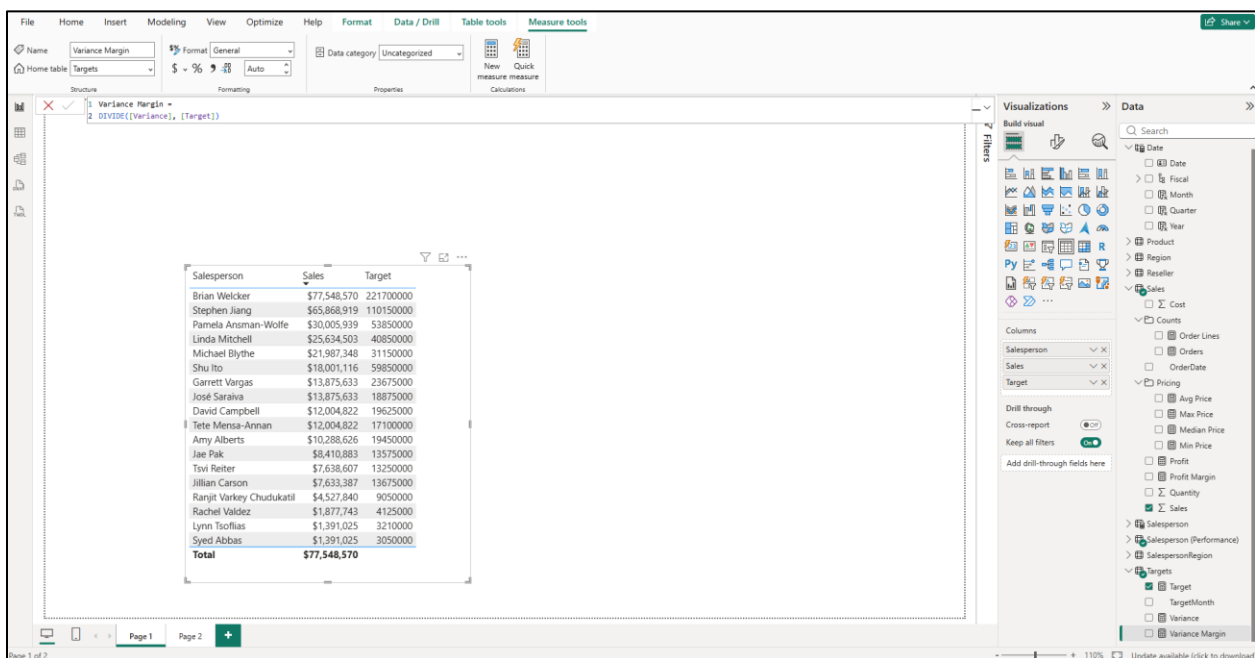
Order Lines =
COUNTROWS(Sales)

>> Targets table: Target measure definition
-----
Target =
IF(
    HASONEVALUE('Salesperson (Performance)'[Salesperson]),
    SUM(Targets[TargetAmount])
)

>> Targets table: Variance measure definition
-----
Variance =
IF(
    HASONEVALUE('Salesperson (Performance)'[Salesperson]),
    SUM(Sales[Sales]) - [Target]
)

>> Targets table: Variance Margin measure definition
-----
Variance Margin =
DIVIDE([Variance], [Target])

```



- Format the new measures: Go to the Measure Tools tab and make your selection in the Format drop down list. Format the Variance measure as whole number (0 decimals). Format the Variance Margin measure as percentage with 2 decimals.

- Add measures to the table: Drag the Variance measure and the Variance Margin measure to the table visual. Resize to see all columns and rows.

Salesperson	Sales	Target	Variance	Variance Margin
Brian Welcker	\$77,548,570	221700000	-144,151,430	-0.65
Stephen Jiang	\$65,868,919	110150000	-44,281,081	-0.40
Pamela Anzaman-Wolfe	\$30,005,939	53850000	-23,844,061	-0.44
Linda Mitchell	\$25,634,503	40850000	-15,215,497	-0.37
Michael Blythe	\$21,987,348	31150000	-9,162,652	-0.29
Shu Ito	\$18,001,116	59850000	-41,848,884	-0.70
Garrett Vargas	\$13,875,633	23675000	-9,799,367	-0.41
José Saraiwa	\$13,875,633	18875000	-4,999,367	-0.26
David Campbell	\$12,004,822	19625000	-7,620,178	-0.39
Tete Mensa-Annan	\$12,004,822	17100000	-5,095,178	-0.30
Amy Alberts	\$10,288,626	19450000	-9,161,374	-0.47
Jae Pak	\$8,410,883	13575000	-5,164,117	-0.38
Tovi Reiter	\$7,638,607	13250000	-5,611,393	-0.42
Jillian Carlson	\$7,633,387	13675000	-6,041,613	-0.44
Ranjit Varkey Chudukatil	\$4,527,840	9050000	-4,522,160	-0.50
Rachel Valdez	\$1,877,743	4125000	-2,247,257	-0.54
Lynn Tsoflias	\$1,391,025	3210000	-1,818,975	-0.57
Syed Abbas	\$1,391,025	3050000	-1,658,975	-0.54
Total	\$77,548,570			

- Reset the Data pane: At the top right of the Data pane, click >> to collapse it and expand again. This refreshes the content.
- Tables that have visible measures appear at the top.

9. Reflection

After working through this lab, I now understand that DAX isn't a programming language but a formula language in Power BI designed to build calculations over tables and columns in a semantic model. This lab shows you how to write DAX calculations to apply the right business logic for your reports. The main purpose of DAX is to support advanced analytics by allowing users to define custom calculations directly in the semantic model, making them reusable, consistent, and fully dynamic across all visuals and reports. I learned that there are three different types of DAX calculations. Calculated tables change the model's structure, calculated columns add row-level logic, and measures perform aggregations across the model. Simple measures use basic aggregations to produce fast visuals. Complex measures are far more flexible and customizable for real-world analytics, use complex DAX calculations, and enable interactive dashboards and reports that respond to slicers and filters. This lab teaches you how to take control of how Power BI calculates your metrics. Instead of relying on Power BI's automatic aggregations, you learn to define the business logic behind each calculation. You learn to build measures that are accurate, consistent and align with reporting needs.